

Modulo 10: Teste e Qualidade de Software

Aula 01: Introdução à Qualidade de Software

Mudança de Paradigma no Desenvolvimento Full Stack

O que significa **"Funcionar"**?



Mudança de Paradigma no Desenvolvimento Full Stack

Passado: Entregar funcionalidades dentro do prazo era o único indicador de sucesso.

Presente: O software precisa ser consistente, seguro e escalável.

A qualidade não é uma etapa final; é um atributo construído a cada commit, escolha de arquitetura e linha de código escrita.

Os Quatro Pilares da Qualidade de Software

 **Confiabilidade:** Manter o desempenho sob condições estabelecidas (ex: transações íntegras no banco de dados).

 **Usabilidade:** Facilidade de interação, front-end intuitivo e respeito à acessibilidade (UX).

 **Eficiência:** Otimização de recursos (tamanho do bundle JS no front-end, velocidade de queries SQL no back-end).

 **Manutenibilidade:** Facilidade de modificar, corrigir bugs ou adaptar o código com baixo acoplamento e testes.

O Ciclo de Vida do Desenvolvimento (SDLC) e a Qualidade

Qualidade deve ser um processo contínuo e preventivo.

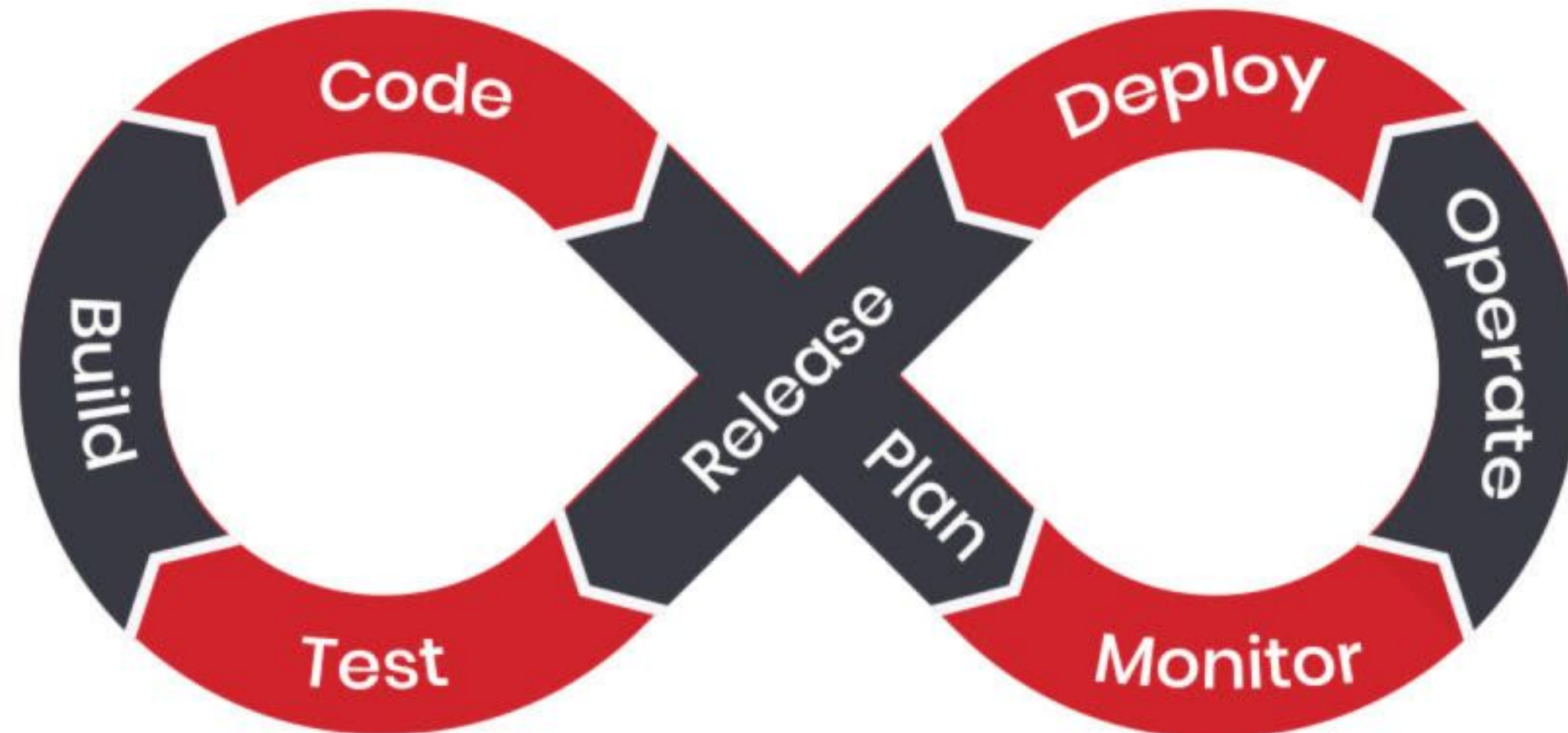
O ciclo não começa na escrita do código, mas no levantamento de requisitos.

Shift-Left Testing: A estratégia de trazer os testes e as validações para o início do ciclo de vida do projeto.





SHIFT LEFT TESTING



A Lei Exponencial do Custo do Bug

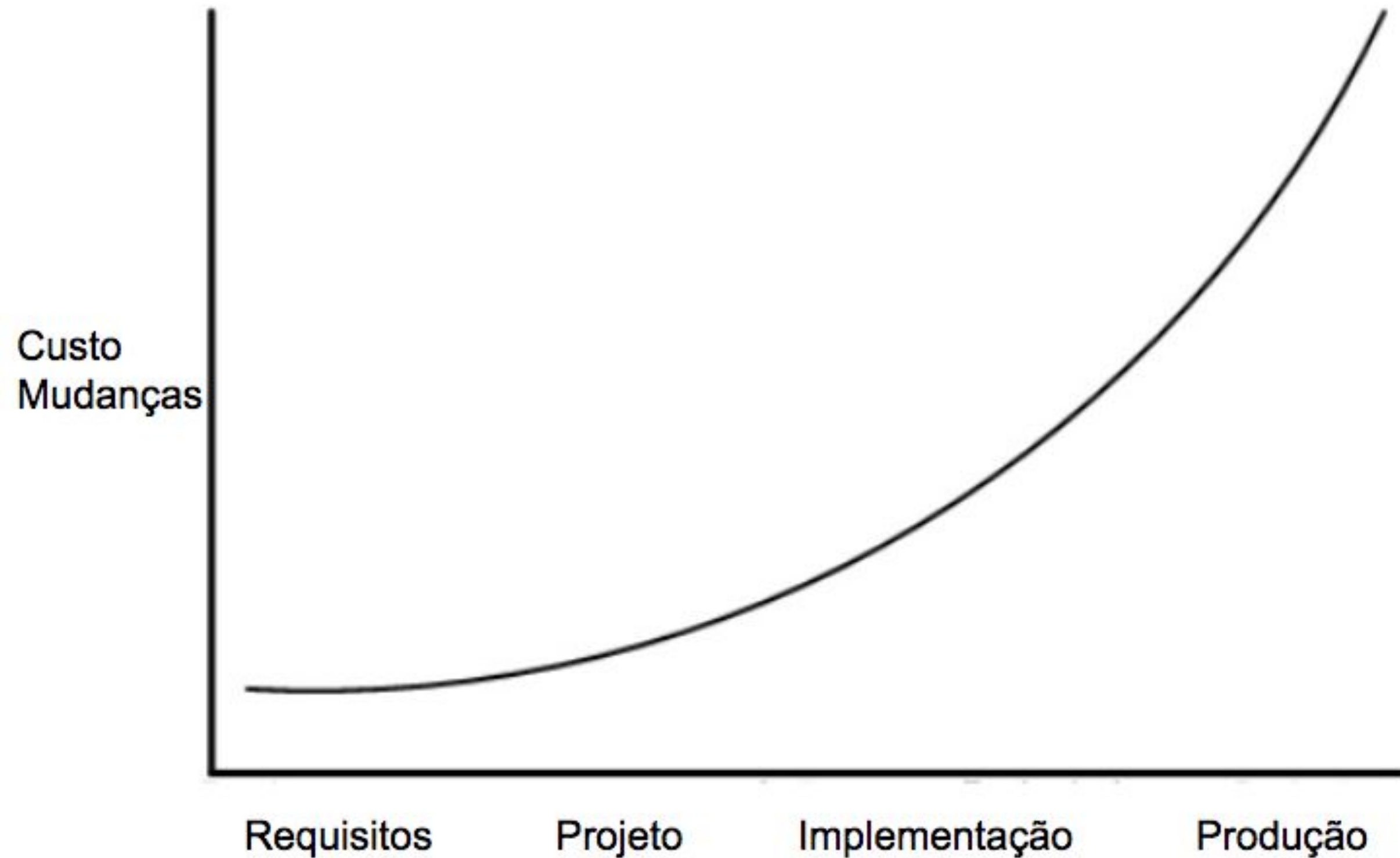
Avançar sem testes acumula **Débito Técnico**.

O custo para corrigir um erro aumenta exponencialmente a cada fase:

Requisitos / Desenho: Custo mínimo (reescrever um documento).

Codificação: Custo baixo (alguns minutos de refatoração).

Produção (Deploy): Custo altíssimo (envolve vazamento de dados, rollbacks emergenciais e insatisfação de clientes).



Intervalo



Duração 15 Min.

Duração > 19:30 – 19:45

Mão na Massa — Configurando o Ambiente

Objetivo: Identificar violações dos pilares de Eficiência e Manutenibilidade em tempo de execução.

Pré-requisitos instalados: Python 3.13 e VS Code.

❖ **Instruções Iniciais:**

- Criem uma pasta chamada **aula01-qualidade** no computador.
- Abram a pasta no VS Code.
- Criem um arquivo chamado **performance.py**.

Processamento de Grande Volume de Dados

```
import time

def rodar_laboratorio():
    tamanho_giga = 15000
    tempo_inicial = time.time()

    # Simulando uma busca ineficiente de dados (O(N^2))
    resultado = []
    for i in range(tamanho_giga):
        for j in range(tamanho_giga):
            if i == j:
                resultado.append(i)

    tempo_final = time.time()
    tempo_execucao = tempo_final - tempo_inicial

    print(f"Tempo de Execução: {tempo_execucao:.4f} segundos")
    print("Total de itens processados:", len(resultado))

rodar_laboratorio()
```


Desafio Full Stack: Aplicando QA e QC

Controle de Qualidade (QC): Nosso teste manual/executável provou que o sistema está lento.

Garantia da Qualidade (QA): Como intervir no processo de escrita do código para garantir o pilar de Eficiência e Manutenibilidade?

Missão dos Alunos: Reescrever o algoritmo eliminando o laço interno (for j).

O script precisa gerar o mesmo resultado final na lista, mas usando apenas uma estrutura de repetição linear simples.

Resolução e Discussão Técnica

```
import time

def rodar_laboratorio_otimizado():
    tamanho_giga = 15000
    tempo_inicial = time.time()

    # Abordagem linear e performática (O(N))
    resultado = []
    for i in range(tamanho_giga):
        resultado.append(i)

    tempo_final = time.time()
    tempo_execucao = tempo_final - tempo_inicial

    print(f"Tempo de Execução Otimizado: {tempo_execucao:.4f} segundos")
    print("Total de itens processados:", len(resultado))

rodar_laboratorio_otimizado()
```

Padronização Internacional: A Antiga ISO 9126

ISO 9126: Modelo pioneiro de avaliação de produto de software.

- ❖ Divisão clássica em características internas e externas:
 - Funcionalidade e Confiabilidade.
 - Usabilidade e Eficiência.

Manutenibilidade e Portabilidade.

Por que mudou? Tornou-se obsoleta diante da evolução da web moderna, cloud e microsserviços.



O Padrão Moderno: ISO 25000 (SQuaRE)

SQuaRE: Software Product Quality Requirements and Evaluation.
Evolução direta e robusta da antiga ISO 9126, totalmente adaptada para softwares modernos.

- ❖ Avalia subcaracterísticas cruciais:
 - Adequação funcional.
 - Compatibilidade (ex: trocas de dados via JSON/REST entre sistemas).
 - Segurança (Criptografia, autenticação e proteção de dados).

ite

Paix



Duas Frentes de Atuação: QA vs. QC

❖ **Quality Assurance (QA) – Preventivo:**

- Foco nos processos de desenvolvimento.
- Evita que o erro aconteça (Padrões de código, arquiteturas limpas, pipelines de CI/CD).

❖ **Quality Control (QC) – Corretivo:**

- Foco no produto construído.
- Encontra defeitos após a construção (Execução manual ou automatizada de testes na aplicação pronta).

Dinâmica Prática: O Apagão do E-commerce

O Cenário: Um e-commerce local funciona perfeitamente quando acessado por 10 usuários simultâneos.

No horário do almoço, recebe 500 acessos simultâneos no carrinho e cai completamente.

❖ **O Desafio em Grupo:**

- Qual pilar de qualidade da ISO 25000 foi violado? Justifique.
- Isso foi uma falha no processo de QA ou no produto final (QC)?
- Como evitar isso no ciclo de vida de desenvolvimento?